

prompt de integração do turbo vision com o console gráfico

Meu idioma é português.

Ontem nós paramos tentando fazer o turbo vision funcionar dentro de um componente TForm do Lazarus. Criamos um Console Gráfico usando com ancestral o componente TCustomControl; Esse componente foi implementado em etapas segundo a natureza do código para separar eventos de cursor, teclado, mouse, captura de tela e vídeo. Cada unit abaixo contem uma função do Console Gráfico: 1 - ConsoleGraphicAbstract_u.pas, //Classe abstrata para o console virtual; 2 - ConsoleGraphicBuffer.pas // Gerencia dois buffer de tela o atual e o anterior 3 - ConsoleGraphicCursor.pas // Gerencia o tratamento do cursor; 4 - ConsoleGraphicMouse.pas // Gerencia os enventos de mouse; 5 - ConsoleGraphicScreenCapture.pas // Gerencia a área de transferência; 6 - ConsoleGraphicVideo.pas // Classe responsável para renderizar o buffer na tela.

A unit FrameLCL abaixo substitui a unit drivers.pas e é ancestral de TFrame que contem um TConsoleGraphicVideo que substitui a api windows. A mesma executa HandleEvent da Aplicação turbo Vision toda vez que um evento de mouse e teclado ocorrer.

No momento estamos adaptando o turbo vision para que ao invés de usar a api windows use o console gráfico.

Identifiquei que o turbo vision não está recebendo os evento de mouseUp e MouseMove por isso quero que você se concentre neste trabalho

```
{: Unit **@name** é usada para gerenciamento de eventos de mouse na  
interface gráfica de console, integrando-se à unit ConsoleGraphic.pas.  
Fornece funcionalidades para detectar, simular e processar eventos de  
mouse, incluindo cliques, movimentos e cliques duplos, com suporte a  
manipuladores de eventos personalizados.  
@author(Analista: Paulo Pacheco)  
@author(Programadores: Paulo Pacheco, Grok, ChatGPT, Qwen)  
@version(0.0.2.8)  
@date(29/07/2025)  
@time(10:35:00 hs - Horário de Brasília)  
@status(Funcional - Adicionado método DoHandlerEvent para  
processamento genérico de eventos de mouse, corrigidos erros de tipo e  
visibilidade.)  
}  
unit ConsoleGraphicMouse;  
  
{ $IFDEF FPC  
  { $MODE Delphi } { $H+ }  
{ $ELSE  
  { $DEFINE DELPHI }  
{ $ENDIF }  
  
interface
```

```

uses
  Classes, SysUtils, Graphics, Controls, ConsoleGraphicBuffer, Forms,
  Use32,
  Math, ConsoleGraphicKeyEvent, ConsoleGraphicAbstract;

const
  mbLeftButton = $01;
  mbRightButton = $02;
  mbMiddleButton = $03;

type
  TMouseEventHandler = function(EventType: SmallWord; Button:
    TMouseButton; Shift: TShiftState; X, Y: LongInt): Boolean of object;

  { TConsoleGraphicMouse }

  TConsoleGraphicMouse = class(TConsoleGraphicKeyEvent)
  private
    FLastMousePos : TSysPointMouse;
  var
    fMouseCellX, fMouseCellY: Integer;
    FLastMouseX, FLastMouseY: Integer;

    FMouseEvents: TList;
    FMouseHandlers: TList;
    FMouseEventAvailable: Boolean;
    FMouseVisible: Boolean;
    FMouseX: Integer;
    FMouseY: Integer;
    FMouseButtons: Byte;
    FLastButtons: Byte;
    FDownButtons: Byte;
    FDownWhere: USE32.TPoint;
    FDownTicks: SmallWord;
    FAutoTicks: SmallWord;
    FAutoDelay: SmallWord;
    FDoubleDelay: SmallWord;
    FLastDouble: Boolean;
    FRepeatDelay: SmallWord;
    FMouseReverse: Boolean;
    function SysPointMouseToTPoint(const SysPoint: TSysPointMouse):
      USE32.TPoint;
  public
    function SysTVDetectMouse: Longint; override;
    procedure SysTVShowMouse; override;
    procedure SysTVHideMouse; override;
    procedure SysTVCreateMouse(var X, Y: Integer); override;
    procedure SysTVDestroyMouse(Close: Boolean); override;
    function SysTVGetMouseEvent(var Event: TSysMouseEvent): Boolean;
  override;
    procedure SysTVUpdateMouseWhere(var X, Y: Integer); override;
    procedure SimulateMouseDown(Button: TMouseButton; Shift: TShiftState;
      X, Y: LongInt); override;
    procedure MouseMove(Shift: TShiftState; X, Y: LongInt); override;
    constructor Create(AOwner: TComponent); override;

```

```

constructor Create(AOwner: TComponent); override;
destructor Destroy; override;
function GetMouseEventCount: Integer; override;
procedure GetMouseEvent(var Event: TEvent);
procedure MouseDown(Button: TMouseButton; Shift: TShiftState; X, Y:
LongInt); override;
procedure DoMouseHandler(EventType: SmallWord; Button: TMouseButton;
Shift: TShiftState; X, Y: LongInt); virtual;
function SetMouseEventHandler(HandlerRoutine: TMouseEventHandler;
Add: Boolean): Boolean; virtual;

    //procedure DoHandlerEvent(var Event: TEvent); override;
end;

implementation

{ TConsoleGraphicMouse }

function TConsoleGraphicMouse.SysPointMouseToTPoint(const SysPoint:
TSysPointMouse): USE32.TPoint;
begin
    Result.X := SysPoint.X div Max(1, FCellWidth);
    Result.Y := SysPoint.Y div Max(1, FCellHeight);
end;

constructor TConsoleGraphicMouse.Create(AOwner: TComponent);
begin
    inherited Create(AOwner);
    FMouseEvents := TList.Create;
    FMouseHandlers := TList.Create;
    FMouseEventAvailable := False;
    FMouseVisible := False;
    FMouseX := 0;
    FMouseY := 0;
    FMouseButtons := 0;
    FLastButtons := 0;
    FDownButtons := 0;
    FDownWhere.X := 0;
    FDownWhere.Y := 0;
    FDownTicks := 0;
    FAutoTicks := 0;
    FAutoDelay := 10; // 550ms (10 ticks * 55ms)
    FDoubleDelay := 8; // ~440ms
    FLastDouble := False;
    FRepeatDelay := 10; // 550ms
    FMouseReverse := False;
    //writeln('TConsoleGraphicMouse.Create: Sistema de mouse
inicializado');
end;

destructor TConsoleGraphicMouse.Destroy;
var
    I: Integer;
    P: PSysMouseEvent;
    M: ^TMethod;

```

```
begin
  for I := 0 to FMouseEvents.Count - 1 do
    begin
      P := FMouseEvents[I];
      if P <> nil then
        Dispose(P);
      end;
    FMouseEvents.Free;

    for I := 0 to FMouseHandlers.Count - 1 do
      begin
        M := FMouseHandlers[I];
        if M <> nil then
          Dispose(M);
        end;
      FMouseHandlers.Free;

      inherited Destroy;
    end;

    function TConsoleGraphicMouse.SysTVDetectMouse: Longint;
    begin
      Result := 1; // Simula suporte ao mouse
      //writeln('SysTVDetectMouse: Suporte ao mouse detectado (simulado)');
    end;

    procedure TConsoleGraphicMouse.SysTVShowMouse;
    begin
      FMouseVisible := True;
      //writeln('SysTVShowMouse: Cursor do mouse visível');
    end;

    procedure TConsoleGraphicMouse.SysTVHideMouse;
    begin
      FMouseVisible := False;
      //writeln('SysTVHideMouse: Cursor do mouse oculto');
    end;

    procedure TConsoleGraphicMouse.SysTVCreateMouse(var X, Y: Integer);
    begin
      FMouseX := X;
      FMouseY := Y;
      FMouseVisible := True;
      //writeln(Format('SysTVCreateMouse: Cursor do mouse criado em X=%d,
Y=%d', [X, Y]));
    end;

    procedure TConsoleGraphicMouse.SysTVDestroyMouse(Close: Boolean);
    begin
      FMouseEvents.Clear;
      FMouseEventAvailable := False;
      FMouseVisible := False;
      //writeln('SysTVDestroyMouse: Sistema de mouse destruído');
    end;
```

```
function TConsoleGraphicMouse.SysTVGetMouseEvent(var Event:
TSysMouseEvent): Boolean;
var
  P: PSysMouseEvent;
  WaitStart: TDateTime;
begin
  WaitStart := Now;
  while not FMouseEventAvailable do
  begin
    if (Now - WaitStart) * 24 * 60 * 60 * 1000 > 1 then
    begin
      //writeln('SysTVGetMouseEvent: Nenhum evento de mouse disponível
(timeout)');
      Result := False;
      Exit;
    end;
    Application.ProcessMessages;
  end;

  if FMouseEvents.Count > 0 then
  begin
    P := FMouseEvents[0];
    Event := P^;
    FMouseEvents.Delete(0);
    Dispose(P);
    FMouseEventAvailable := FMouseEvents.Count > 0;
    //writeln(Format('SysTVGetMouseEvent: Evento de mouse obtido - X=%d,
Y=%d, Buttons=%d', [Event.smePos.X, Event.smePos.Y, Event.smeButtons]));
    Result := True;
  end
  else
  begin
    //writeln('SysTVGetMouseEvent: Fila de eventos vazia após espera');
    FMouseEventAvailable := False;
    Result := False;
  end;
end;

procedure TConsoleGraphicMouse.SysTVUpdateMouseWhere(var X, Y: Integer);
begin
  FMouseX := X;
  FMouseY := Y;
  //writeln(Format('SysTVUpdateMouseWhere: Posição do mouse atualizada
para X=%d, Y=%d', [X, Y]));
end;

procedure TConsoleGraphicMouse.SimulateMouseDown(Button: TMouseButton;
Shift: TShiftState; X, Y: LongInt);
var
  Event: PSysMouseEvent;
  //MouseCellX, MouseCellY: Integer;
  Ev: TEvent;
begin
```

```
fMouseCellX := X div Max(1, FCellWidth);
fMouseCellY := Y div Max(1, FCellHeight);

if (fMouseCellX >= 0) and (fMouseCellX < Cols) and
  (fMouseCellY >= 0) and (fMouseCellY < Rows) then
begin
  SysGotoXY(fMouseCellX, fMouseCellY);
  ActivateBlinkingCursor;
  if IsControlVisible and Enabled and HandleAllocated then
  begin
    try
      SetFocus;
      //writeln(Format('SimulateMouseDown: Foco definido para o
controle em CellX=%d, Celly=%d (PixelX=%d, PixelY=%d)', [fMouseCellX,
fMouseCellY, X, Y]));
    except
      on E: EInvalidOperation do
      begin
        //writeln('SimulateMouseDown: Falha ao definir foco - ' +
E.Message);
      end;
      on E: Exception do
      begin
        //writeln('SimulateMouseDown: Erro inesperado ao definir foco -
' + E.Message);
      end;
    end;
  end
else
  begin
    //writeln(Format('SimulateMouseDown: Não foi possível definir foco
- Visible=%s, Enabled=%s, HandleAllocated=%s',
    //[[BoolToStr(IsControlVisible, True), BoolToStr(Enabled, True),
BoolToStr(HandleAllocated, True)]]);
  end;
end
else
  begin
    //writeln(Format('SimulateMouseDown: Clique fora dos limites -
CellX=%d, Celly=%d, Cols=%d, Rows=%d', [fMouseCellX, fMouseCellY, Cols,
Rows]));
  end;

New(Event);
Event^.smeTime := GetTickCount64;
Event^.smePos.X := X;
Event^.smePos.Y := Y;
Event^.smeButtons := 0;
case Button of
  mbLeft: Event^.smeButtons := mbLeftButton;
  mbRight: Event^.smeButtons := mbRightButton;
  mbMiddle: Event^.smeButtons := mbMiddleButton;
end;

FMouseEvents.Add(Event);
FMouseEventAvailable := True;
```

```
FMouseEventAvailable := True;
//writeLn(Format('SimulateMouseDown: Evento de clique gerado -
Button=%d, X=%d, Y=%d', [Event^.smeButtons, X, Y]));

Ev.What := evMouseDown;
Ev.Buttons := Event^.smeButtons;
Ev.Double := FLastDouble;
Ev.Where.X := fMouseCellX;
Ev.Where.Y := fMouseCellY;
DoHandlerEvent(Ev); //Executa os eventos registros.
end;

function TConsoleGraphicMouse.GetMouseEventCount: Integer;
begin
    Result := FMouseEvents.Count;
end;

procedure TConsoleGraphicMouse.MouseMove(Shift: TShiftState; X, Y:
LongInt);
var
    MouseCellX, MouseCellY: Integer;
    Event: TEvent;
    SysMouseEvent: TSysMouseEvent;
begin
    inherited MouseMove(Shift, X, Y);
    MouseCellX := X div Max(1, FCellWidth);
    MouseCellY := Y div Max(1, FCellHeight);

    // Ignorar eventos redundantes
    if (MouseCellX = FLastMouseX) and (MouseCellY = FLastMouseY) then
    begin
        //WriteLn(Format('MouseMove: Ignorado - Posição redundante X=%d,
Y=%d', [MouseCellX, MouseCellY]));
        Exit;
    end;

    FLastMouseX := MouseCellX;
    FLastMouseY := MouseCellY;

    // Obter estado dos botões do sistema
    if SysTVGetMouseEvent(SysMouseEvent) then
        FMouseButtons := SysMouseEvent.smeButtons
    else
        FMouseButtons := 0;
    if (FMouseButtons = 0) then
    begin
        //WriteLn(Format('MouseMove: Ignorado - Botão do mouse igual a zero',
[MouseCellX, MouseCellY]));
        Exit;
    end;

    Cursor := crArrow;
    if (MouseCellX >= 0) and (MouseCellX < Cols) and (MouseCellY >= 0) and
(MouseCellY < Rows) then
    begin
        if IsInSelection(MouseCellX, MouseCellY) then
```

```
        Cursor := crIBeam;
    end;

    Event.What := evMouseMove;
    Event.Buttons := FMouseButtons; // Propagar estado atual dos botões
    Event.Double := FLastDouble;
    Event.Where.X := MouseCellX;
    Event.Where.Y := MouseCellY;
    DoHandlerEvent(Event); //Executa os eventos registros.

    WriteLn(Format('MouseMove: Evento gerado - X=%d, Y=%d, Buttons=%d,
CursorType=%s',
        [MouseCellX, MouseCellY, Event.Buttons, CursorToString(Cursor)]));
end;

function CursorToString(Cursor: TCursor): string;
begin
    case Cursor of
        crArrow: Result := 'crArrow';
        crIBeam: Result := 'crIBeam';
        crHandPoint: Result := 'crHandPoint';
        else Result := 'crUnknown';
    end;
end;

//procedure TConsoleGraphicMouse.GetMouseEvent(var Event: TEvent);
//var
// SysMouseEvent: TSysMouseEvent;
// CurTicks: SmallWord;
// B: Byte;
//
//
// procedure StoreEvent(MouWhat: SmallWord);
// begin
//     FLastButtons := FMouseButtons;
//     FMouseX := SysMouseEvent.smePos.X;
//     FMouseY := SysMouseEvent.smePos.Y;
//     with Event do
//     begin
//         What := MouWhat;
//         Buttons := FMouseButtons;
//         Double := FLastDouble;
//         Where := SysPointMouseToTPoint(SysMouseEvent.smePos);
//     end;
//     SysTVUpdateMouseWhere(FMouseX, FMouseY);
//     //WriteLn(Format('GetMouseEvent: Evento %d gerado - CellX=%d,
CellY=%d, Buttons=%d, Double=%d',
//     // [MouWhat, Event.Where.X, Event.Where.Y, Event.Buttons,
Ord(FLastDouble)]));
// end;
//begin
// Application.ProcessMessages; // Mantém pausa para evitar consumo
excessivo de CPU
// Event.What := evNothing; // Inicializa evento como nulo
// Event.Where.X := 0;
```

```
// Event.Where.Y := 0;
// if not SysTVGetMouseEvent(SysMouseEvent) then
// begin
//   FMouseButtons := FLastButtons;
//   SysMouseEvent.smePos.X := FMouseX;
//   SysMouseEvent.smePos.Y := FMouseY;
//   SysMouseEvent.smeTime := GetTickCount64;
//   Event.Where.X := FMouseX div Max(1, FCellWidth); // Ex.: 8 pixels
//   por célula
//   Event.Where.Y := FMouseY div Max(1, FCellHeight); // Ex.: 16 pixels
//   por célula
//   //WriteLn(Format('GetMouseEvent: Nenhum evento disponível, usando
//   última posição - CellX=%d, Celly=%d',
//   //   // [Event.Where.X, Event.Where.Y]));
//   // Removido: if FMouseButtons <> 0 then StoreEvent(evMouseDown);
//   // Alinhado ao DOS: Definir Event.What := evNothing explicitamente
//   para ausência de eventos
//   Event.What := evNothing; // Adicionado para consistência com a
//   versão DOS
// end
// else
// begin
//   //WriteLn(Format('SysTVGetMouseEvent: Pos.X=%d, Pos.Y=%d,
//   Buttons=%d',
//   //   // [SysMouseEvent.smePos.X, SysMouseEvent.smePos.Y,
//   SysMouseEvent.smeButtons]));
//   if FMouseReverse then
//   begin
//     B := 0;
//     if (SysMouseEvent.smeButtons and mbLeftButton) <> 0 then
//       Inc(B, mbRightButton);
//     if (SysMouseEvent.smeButtons and mbRightButton) <> 0 then
//       Inc(B, mbLeftButton);
//     SysMouseEvent.smeButtons := B;
//   end;
//   FMouseButtons := SysMouseEvent.smeButtons;
//   CurTicks := SysMouseEvent.smeTime div 55;
//   if (FLastButtons <> 0) and (FMouseButtons = 0) then
//   begin
//     StoreEvent(evMouseUp);
//     //WriteLn('GetMouseEvent: evMouseUp gerado');
//   end
//   else if FLastButtons = FMouseButtons then
//   begin
//     if (SysMouseEvent.smePos.Y <> FMouseY) or (SysMouseEvent.smePos.X
//     <> FMouseX) then
//     begin
//       StoreEvent(evMouseMove);
//       //WriteLn('GetMouseEvent: evMouseMove gerado');
//     end
//     else if (FMouseButtons <> 0) and ((CurTicks - FAutoTicks) >=
//     FAutoDelay) then
//     begin
//       FAutoTicks := CurTicks;
//       FAutoDelay := 1;
//     end
//   end
// end
```

```
//      StoreEvent(evMouseAuto);
//      //WriteLn('GetMouseEvent: evMouseAuto gerado');
//      end
//      else
//      begin
//          StoreEvent(evNothing);
//          //WriteLn('GetMouseEvent: evNothing gerado (mesma posição, sem
auto-repeat)');
//      end;
//      end
//      else
//      begin
//          FLastDouble := False;
//          if (FMouseButtons = FDownButtons) and (SysMouseEvent.smePos.Y =
FDownWhere.Y) and
//              (SysMouseEvent.smePos.X = FDownWhere.X) and ((CurTicks -
FDownTicks) < FDoubleDelay) then
//              begin
//                  FLastDouble := True;
//                  //WriteLn('GetMouseEvent: Duplo clique detectado');
//              end;
//          FDownButtons := FMouseButtons;
//          FDownWhere := SysPointMouseToTPoint(SysMouseEvent.smePos);
//          FDownTicks := CurTicks;
//          FAutoTicks := CurTicks;
//          FAutoDelay := FRepeatDelay;
//          StoreEvent(evMouseDown);
//          //WriteLn('GetMouseEvent: evMouseDown gerado');
//      end;
//  end;
//end;

//procedure TConsoleGraphicMouse.GetMouseEvent(var Event: TEvent);
//var
//  SysMouseEvent: TSysMouseEvent;
//  CurTicks: SmallWord;
//  B: Byte;
//
//  procedure StoreEvent(MouWhat: SmallWord);
//  begin
//      FLastButtons := FMouseButtons;
//      FMouseX := SysMouseEvent.smePos.X;
//      FMouseY := SysMouseEvent.smePos.Y;
//      with Event do
//      begin
//          What := MouWhat;
//          Buttons := FMouseButtons;
//          Double := FLastDouble;
//          Where := SysPointMouseToTPoint(SysMouseEvent.smePos);
//      end;
//      SysTVUpdateMouseWhere(FMouseX, FMouseY);
//      //WriteLn(Format('GetMouseEvent: Evento %d gerado - CellX=%d,
CellY=%d, Buttons=%d, Double=%d',
//          // [MouWhat, Event.Where.X, Event.Where.Y, Event.Buttons,
Ord(FLastDouble)]));
//  end;
```

```
// end,
//begin
// Application.ProcessMessages; // Mantém pausa para evitar consumo
excessivo de CPU
// Event.What := evNothing; // Inicializa evento como nulo
// Event.Where.X := 0;
// Event.Where.Y := 0;
// if not SysTVGetMouseEvent(SysMouseEvent) then
// begin
//   FMouseButtons := FLastButtons;
//   SysMouseEvent.smePos.X := FMouseX;
//   SysMouseEvent.smePos.Y := FMouseY;
//   SysMouseEvent.smeTime := GetTickCount64;
//   Event.Where.X := FMouseX div Max(1, FCellWidth); // Ex.: 8 pixels
por célula
//   Event.Where.Y := FMouseY div Max(1, FCellHeight); // Ex.: 16 pixels
por célula
//   Event.What := evNothing; // Adicionado para consistência com a
versão DOS
// end
// else
// begin
//   if FMouseReverse then
// begin
//   B := 0;
//   if (SysMouseEvent.smeButtons and mbLeftButton) <> 0 then
//     Inc(B, mbRightButton);
//   if (SysMouseEvent.smeButtons and mbRightButton) <> 0 then
//     Inc(B, mbLeftButton);
//   SysMouseEvent.smeButtons := B;
// end;
//   FMouseButtons := SysMouseEvent.smeButtons;
//   CurTicks := (SysMouseEvent.smeTime div 55) mod 65536; // Corrigido
para evitar overflow
//   if (FLastButtons <> 0) and (FMouseButtons = 0) then
// begin
//   StoreEvent(evMouseUp);
//   //WriteLn('GetMouseEvent: evMouseUp gerado');
// end
//   else if FLastButtons = FMouseButtons then
// begin
//   if (SysMouseEvent.smePos.Y <> FMouseY) or (SysMouseEvent.smePos.X
<> FMouseX) then
// begin
//   StoreEvent(evMouseMove);
//   //WriteLn('GetMouseEvent: evMouseMove gerado');
// end
//   else if (FMouseButtons <> 0) and ((CurTicks - FAutoTicks) >=
FAutoDelay) then
// begin
//   FAutoTicks := CurTicks;
//   FAutoDelay := 1;
//   StoreEvent(evMouseAuto);
//   //WriteLn('GetMouseEvent: evMouseAuto gerado');
// end
//   else
```

```
//      begin
//          StoreEvent(evNothing);
//          //WriteLn('GetMouseEvent: evNothing gerado (mesma posição, sem
//auto-repeat)');
//      end;
//  end
//  else
//      begin
//          FLastDouble := False;
//          if (FMouseButtons = FDownButtons) and (SysMouseEvent.smePos.Y =
//FDownWhere.Y) and
//          (SysMouseEvent.smePos.X = FDownWhere.X) and ((CurTicks -
//FDownTicks) < FDoubleDelay) then
//              begin
//                  FLastDouble := True;
//                  //WriteLn('GetMouseEvent: Duplo clique detectado');
//              end;
//              FDownButtons := FMouseButtons;
//              FDownWhere := SysPointMouseToTPoint(SysMouseEvent.smePos);
//              FDownTicks := CurTicks;
//              FAutoTicks := CurTicks;
//              FAutoDelay := FRepeatDelay;
//              StoreEvent(evMouseDown);
//              //WriteLn('GetMouseEvent: evMouseDown gerado');
//          end;
//      end;
//end;
//
procedure TConsoleGraphicMouse.GetMouseEvent(var Event: TEvent);
var
    SysMouseEvent: TSysMouseEvent;
    CurTicks: SmallWord;
    IsMenuContext: Boolean;
    LastPos: TSysPointMouse;
begin
    //WriteLn('ConsoleGraphicMouse.GetMouseEvent: Iniciando');
    FillChar(Event, SizeOf(TEvent), 0);
    Event.Where.X := 0;
    Event.Where.Y := 0;
    LastPos := FLastMousePos; // Supondo FLastMousePos: TSysPointMouse na
classe
    if not SysTVGetMouseEvent(SysMouseEvent) then
        begin
            //WriteLn('ConsoleGraphicMouse.GetMouseEvent: Nenhum evento de mouse
disponível');
            Exit;
        end;

    WriteLn('ConsoleGraphicMouse.GetMouseEvent: Evento bruto, X=',
SysMouseEvent.smePos.X,
            ', Y=', SysMouseEvent.smePos.Y, ', Buttons=',
SysMouseEvent.smeButtons,
            ', Time=', SysMouseEvent.smeTime);

    Event.Where.X := SysMouseEvent.smePos.X div FCellWidth;
```

```
Event.Where.Y := SysMouseEvent.smePos.Y div FCellHeight;

if (Event.Where.X < 0) or (Event.Where.X >= Cols) or
  (Event.Where.Y < 0) or (Event.Where.Y >= Rows) then
begin
  WriteLn('ConsoleGraphicMouse.GetMouseEvent: Coordenadas fora dos
limites, X=', Event.Where.X,
          ', Y=', Event.Where.Y);
  Exit;
end;
WriteLn('ConsoleGraphicMouse.GetMouseEvent: Coordenadas convertidas,
X=', Event.Where.X,
        ', Y=', Event.Where.Y);

IsMenuContext := (Event.Where.Y <= 1);
Event.Buttons := SysMouseEvent.smeButtons;

if (SysMouseEvent.smePos.X <> LastPos.X) or (SysMouseEvent.smePos.Y <>
LastPos.Y) then
begin
  if Event.Buttons = 0 then
  begin
    Event.What := evMouseMove;
    WriteLn('ConsoleGraphicMouse.GetMouseEvent: evMouseMove gerado');
  end;
end;

if Event.Buttons <> FMouseButtons then
begin
  if Event.Buttons <> 0 then
    Event.What := evMouseDown
  else
    Event.What := evMouseUp;
  FMouseButtons := Event.Buttons;
  WriteLn('ConsoleGraphicMouse.GetMouseEvent: Evento gerado, What=',
Event.What,
          ', Buttons=', Event.Buttons);
end;

if (FMouseButtons <> 0) and ((CurTicks - FAutoTicks) >= FAutoDelay) and
not IsMenuContext then
begin
  FAutoTicks := CurTicks;
  FAutoDelay := 1;
  Event.What := evMouseAuto;
  WriteLn('ConsoleGraphicMouse.GetMouseEvent: evMouseAuto gerado');
end;

FLastMousePos := SysMouseEvent.smePos;
//ShowMessageEvent(Event);
end;

procedure TConsoleGraphicMouse.MouseDown(Button: TMouseButton; Shift:
```

```
TShiftState; X, Y: LongInt);
var
    Event: TEvent;
begin
    inherited MouseDown(Button, Shift, X, Y);
    SimulateMouseDown(Button, Shift, X, Y);
    DoMouseHandler(evMouseDown, Button, Shift, X, Y);
    Event.What := evMouseDown;
    Event.Buttons := Ord(Button);
    Event.Double := FLastDouble;

    fMouseCellX := X div Max(1, FCellWidth);
    fMouseCellY := Y div Max(1, FCellHeight);

    Event.Where.X := fMouseCellX;
    Event.Where.Y := fMouseCellY;
    DoHandlerEvent(Event); //Executa os eventos registros.
    //writeln(Format('MouseDown: Clique detectado - Button=%d, X=%d, Y=%d',
[Ord(Button), X, Y]));
end;

procedure TConsoleGraphicMouse.DoMouseHandler(EventType: SmallWord;
Button: TMouseButton; Shift: TShiftState; X, Y: LongInt);
var
    I: Integer;
    Handler: TMouseEventHandler;
    M: TMethod;
begin
    for I := 0 to FMouseHandlers.Count - 1 do
    begin
        M := TMethod(FMouseHandlers[I]^);
        Handler := TMouseEventHandler(M);
        if Assigned(Handler) then
            Handler(EventType, Button, Shift, X, Y);
    end;
end;

//procedure TConsoleGraphicMouse.DoHandlerEvent(var Event: TEvent);
//var
// I: Integer;
// Handler: TMouseEventHandler;
// M: TMethod;
// Button: TMouseButton;
//begin
// case Event.Buttons of
//     mbLeftButton: Button := mbLeft;
//     mbRightButton: Button := mbRight;
//     mbMiddleButton: Button := mbMiddle;
//     else Button := mbLeft; // Valor padrão
// end;
//
// for I := 0 to FMouseHandlers.Count - 1 do
// begin
//     M := TMethod(FMouseHandlers[I]^);
//     Handler := TMouseEventHandler(M);
//     if Assigned(Handler) then
```

```
// 1. Assigned(handler) then
// begin
//     Handler(Event.What, Button, [], Event.Where.X, Event.Where.Y);
//     //writeln(Format('DoHandlerEvent: Manipulador chamado -
// EventType=%d, Button=%d, X=%d, Y=%d',
// [Event.What, Ord(Button), Event.Where.X, Event.Where.Y]));
// end;
// end;
//end;

function TConsoleGraphicMouse.SetMouseEventHandler(HandlerRoutine:
TMouseEventHandler; Add: Boolean): Boolean;
var
    M: ^TMethod;
    I: Integer;
begin
    if Add then
    begin
        if Assigned(HandlerRoutine) then
        begin
            for I := 0 to FMouseHandlers.Count - 1 do
            begin
                if (TMethod(FMouseHandlers[I]^).Code =
TMethod(HandlerRoutine).Code) and
                    (TMethod(FMouseHandlers[I]^).Data =
TMethod(HandlerRoutine).Data) then
                begin
                    //writeln('SetMouseEventHandler: Manipulador já registrado');
                    Result := False;
                    Exit;
                end;
            end;
            New(M);
            M^ := TMethod(HandlerRoutine);
            FMouseHandlers.Add(M);
            //writeln('SetMouseEventHandler: Manipulador adicionado');
            Result := True;
        end
        else
        begin
            //writeln('SetMouseEventHandler: Manipulador inválido');
            Result := False;
        end;
    end
    else
    begin
        if Assigned(HandlerRoutine) then
        begin
            for I := 0 to FMouseHandlers.Count - 1 do
            begin
                if (TMethod(FMouseHandlers[I]^).Code =
TMethod(HandlerRoutine).Code) and
                    (TMethod(FMouseHandlers[I]^).Data =
TMethod(HandlerRoutine).Data) then
                begin
                    Dispose(FMouseHandlers[I]);
```

```

        FMouseHandlers.Delete(I);
        //writeln('SetMouseEventHandler: Manipulador removido');
        Result := True;
        Exit;
    end;
end;
//writeln('SetMouseEventHandler: Manipulador não encontrado');
Result := False;
end
else
begin
    //writeln('SetMouseEventHandler: Manipulador nulo');
    Result := False;
end;
end;
end;

end.

```

```

procedure TConsoleGraphicMouse.GetMouseEvent(var Event: TEvent); var SysMouseEvent:
TSysMouseEvent; CurTicks: SmallWord; B: Byte; begin
WriteLn('ConsoleGraphicMouse.GetMouseEvent: Iniciando'); FillChar(Event, SizeOf(TEvent),
0); // Inicializa evento Event.What := evNothing; Event.Where.X := 0; Event.Where.Y := 0;

if not SysTVGetMouseEvent(SysMouseEvent) then begin
WriteLn('ConsoleGraphicMouse.GetMouseEvent: Nenhum evento de mouse disponível');
Event.Where.X := FMouseX div Max(1, FCellWidth); Event.Where.Y := FMouseY div Max(1,
FCellHeight); Event.Buttons := FLastButtons; Exit; end;

WriteLn(Format('ConsoleGraphicMouse.GetMouseEvent: Evento bruto, X=%d, Y=%d,
Buttons=%d, Time=%d', [SysMouseEvent.smePos.X, SysMouseEvent.smePos.Y,
SysMouseEvent.smeButtons, SysMouseEvent.smeTime]));

// Converter coordenadas de pixel para células Event.Where.X := SysMouseEvent.smePos.X
div Max(1, FCellWidth); Event.Where.Y := SysMouseEvent.smePos.Y div Max(1, FCellHeight);

// Verificar limites if (Event.Where.X < 0) or (Event.Where.X >= Cols) or (Event.Where.Y < 0) or
(Event.Where.Y >= Rows) then begin
WriteLn(Format('ConsoleGraphicMouse.GetMouseEvent: Coordenadas fora dos limites,
X=%d, Y=%d', [Event.Where.X, Event.Where.Y])); Event.What := evNothing; Exit; end;

// Ajustar botões para modo reverso, se necessário if FMouseReverse then begin B := 0; if
(SysMouseEvent.smeButtons and mbLeftButton) <> 0 then Inc(B, mbRightButton); if
(SysMouseEvent.smeButtons and mbRightButton) <> 0 then Inc(B, mbLeftButton);
SysMouseEvent.smeButtons := B; end;

// Atualizar estado dos botões FMouseButtons := SysMouseEvent.smeButtons;
Event.Buttons := FMouseButtons; CurTicks := SysMouseEvent.smeTime div 55 mod 65536; //

```

```
// Detectar mudanças de posição (MouseMove) if (SysMouseEvent.smePos.X <>
FLastMousePos.X) or (SysMouseEvent.smePos.Y <> FLastMousePos.Y) then begin Event.What
:= evMouseMove; WriteLn(Format('ConsoleGraphicMouse.GetMouseEvent: evMouseMove
gerado, X=%d, Y=%d', [Event.Where.X, Event.Where.Y])); end;

// Detectar mudanças no estado dos botões if FMouseButtons <> FLastButtons then begin if
FMouseButtons <> 0 then begin // MouseDown Event.What := evMouseDown;
FDownButtons := FMouseButtons; FDownWhere :=
SysPointMouseToTPoint(SysMouseEvent.smePos); FDownTicks := CurTicks; FAutoTicks :=
CurTicks; FAutoDelay := FRepeatDelay; FLastDouble := (FMouseButtons = FDownButtons) and
(SysMouseEvent.smePos.X = FLastMousePos.X) and (SysMouseEvent.smePos.Y =
FLastMousePos.Y) and ((CurTicks - FDownTicks) < FDoubleDelay); Event.Double :=
FLastDouble; WriteLn(Format('ConsoleGraphicMouse.GetMouseEvent: evMouseDown
gerado, Buttons=%d, Double=%d', [FMouseButtons, Ord(FLastDouble)])); end else begin //
MouseUp Event.What := evMouseUp;
WriteLn(Format('ConsoleGraphicMouse.GetMouseEvent: evMouseUp gerado, X=%d, Y=%d',
[Event.Where.X, Event.Where.Y])); end; end else if (FMouseButtons <> 0) and ((CurTicks -
FAutoTicks) >= FAutoDelay) then begin // MouseAuto (repetição de clique) FAutoTicks :=
CurTicks; FAutoDelay := 1; Event.What := evMouseAuto;
WriteLn('ConsoleGraphicMouse.GetMouseEvent: evMouseAuto gerado'); end;

// Atualizar estado FLastButtons := FMouseButtons; FLastMousePos :=
SysMouseEvent.smePos; FMouseX := SysMouseEvent.smePos.X; FMouseY :=
SysMouseEvent.smePos.Y; SysTVUpdateMouseWhere(FMouseX, FMouseY);

// Chamar manipuladores registrados if Event.What <> evNothing then
DoHandlerEvent(Event); end;

procedure TConsoleGraphicMouse.MouseMove(Shift: TShiftState; X, Y: LongInt); var
MouseCellX, MouseCellY: Integer; Event: TEvent; begin inherited MouseMove(Shift, X, Y);
MouseCellX := X div Max(1, FCellWidth); MouseCellY := Y div Max(1, FCellHeight);

// Ignorar eventos redundantes if (MouseCellX = FLastMouseX) and (MouseCellY =
FLastMouseY) then begin WriteLn(Format('MouseMove: Ignorado - Posição redundante
X=%d, Y=%d', [MouseCellX, MouseCellY])); Exit; end;

FLastMouseX := MouseCellX; FLastMouseY := MouseCellY;

// Atualizar cursor com base na posição Cursor := crArrow; if (MouseCellX >= 0) and
(MouseCellX < Cols) and (MouseCellY >= 0) and (MouseCellY < Rows) then begin if
IsInSelection(MouseCellX, MouseCellY) then Cursor := crIBeam; end;

// Gerar evento de movimento Event.What := evMouseMove; Event.Buttons :=
FMouseButtons; Event.Double := FLastDouble; Event.Where.X := MouseCellX; Event.Where.Y
:= MouseCellY; DoHandlerEvent(Event);
```

```
WriteLn(Format('MouseMove: Evento gerado - X=%d, Y=%d, Buttons=%d, CursorType=%s',  
[MouseCellX, MouseCellY, Event.Buttons, CursorToString(Cursor)])); end;
```

```
procedure TConsoleGraphicMouse.MouseDown(Button: TMouseButton; Shift: TShiftState; X,  
Y: LongInt); var Event: TEvent; begin inherited MouseDown(Button, Shift, X, Y); fMouseCellX  
:= X div Max(1, FCellWidth); fMouseCellY := Y div Max(1, FCellHeight);
```

```
// Gerar evento de clique SimulateMouseDown(Button, Shift, X, Y); Event.What :=  
evMouseDown; Event.Buttons := Ord(Button); Event.Double := FLastDouble; Event.Where.X  
:= fMouseCellX; Event.Where.Y := fMouseCellY; DoHandlerEvent(Event);
```

```
WriteLn(Format('MouseDown: Clique detectado - Button=%d, X=%d, Y=%d', [Ord(Button), X,  
Y])); end;
```

```
procedure TConsoleGraphicMouse.DoHandlerEvent(var Event: TEvent); var I: Integer;  
Handler: TMouseEventHandler; M: TMethod; Button: TMouseButton; begin // Converter  
Event.Buttons para TMouseButton case Event.Buttons of mbLeftButton: Button := mbLeft;  
mbRightButton: Button := mbRight; mbMiddleButton: Button := mbMiddle; else Button :=  
mbLeft; // Valor padrão end;
```

```
// Chamar manipuladores registrados for I := 0 to FMouseHandlers.Count - 1 do begin M :=  
TMethod(FMouseHandlers[I]^); Handler := TMouseEventHandler(M); if Assigned(Handler)  
then begin Handler(Event.What, Button, [], Event.Where.X, Event.Where.Y);  
WriteLn(Format('DoHandlerEvent: Manipulador chamado - EventType=%d, Button=%d,  
X=%d, Y=%d', [Event.What, Ord(Button), Event.Where.X, Event.Where.Y])); end; end; end;
```